

Web Applications A.Y. 2021-2022
Homework 1 – Server-side Design and Development

Master Degree in Computer Engineering
Master Degree in Cybersecurity
Master Degree in ICT for Internet and Multimedia

Deadline: 24 April, 2022

Group AmuParkWA	Project Amusement Park Web App	
Last Name	First Name	Student Number
Faggioli	Guglielmo	123456

Objectives of the System

The objective of the project, in continuation with the one developed during the database course, is to develop a web interface to manage data for a fictitious amusement ride builder. The project is mainly focused on aspects related to the exchange of the data between users and the back end.

Main Functionalities

The web application is used mainly to allow users to interact with data collected by the fictitious amusement ride builder. Furthermore, the web interface allows to integrate the data with new information. The website is substantially divided into four main areas:

- public area: available to unregistered users and registered users with the “base” role. This area, which also corresponds to the homepage/landing page contains the list of parks for which the amusement ride builder worked for and the list of rides built in each park.
- maintenance area: this area includes three main subpages:
 - inserting and editing maintenance events: used by maintainers to insert or edit maintenance events
 - searching for maintenance events: used to filter through maintenance events and show relevant ones
 - inserting new measurement files: such files are uploaded via web interface and processed offline.
- builder area: the builder area allows to:
 - insert new parks and edit existing ones. to be used when rides have not been deployed before on such park
 - insert new models and edit existing ones: used when the fictitious builder develops a new model
 - insert new rides and edit existing ones.
- administration area: the administration area is accessible only to users with the role “admin”. It contains functionalities to operate over the users in the system. In particular, in this area is possible to:
 - search users via email
 - retrieve information on a specific user based on its email
 - modify the user record by changing their first name, last name and role

Roles can be sorted according to the privileges they grant. In particular, admin > builder > maintainer > base users. This means that a user can access all the areas of the web-site granted by their and lower roles.

The site has an accessory area which allows every user to either register or login to the application. Note that, if the registration is completed successfully the user should be notified via email.

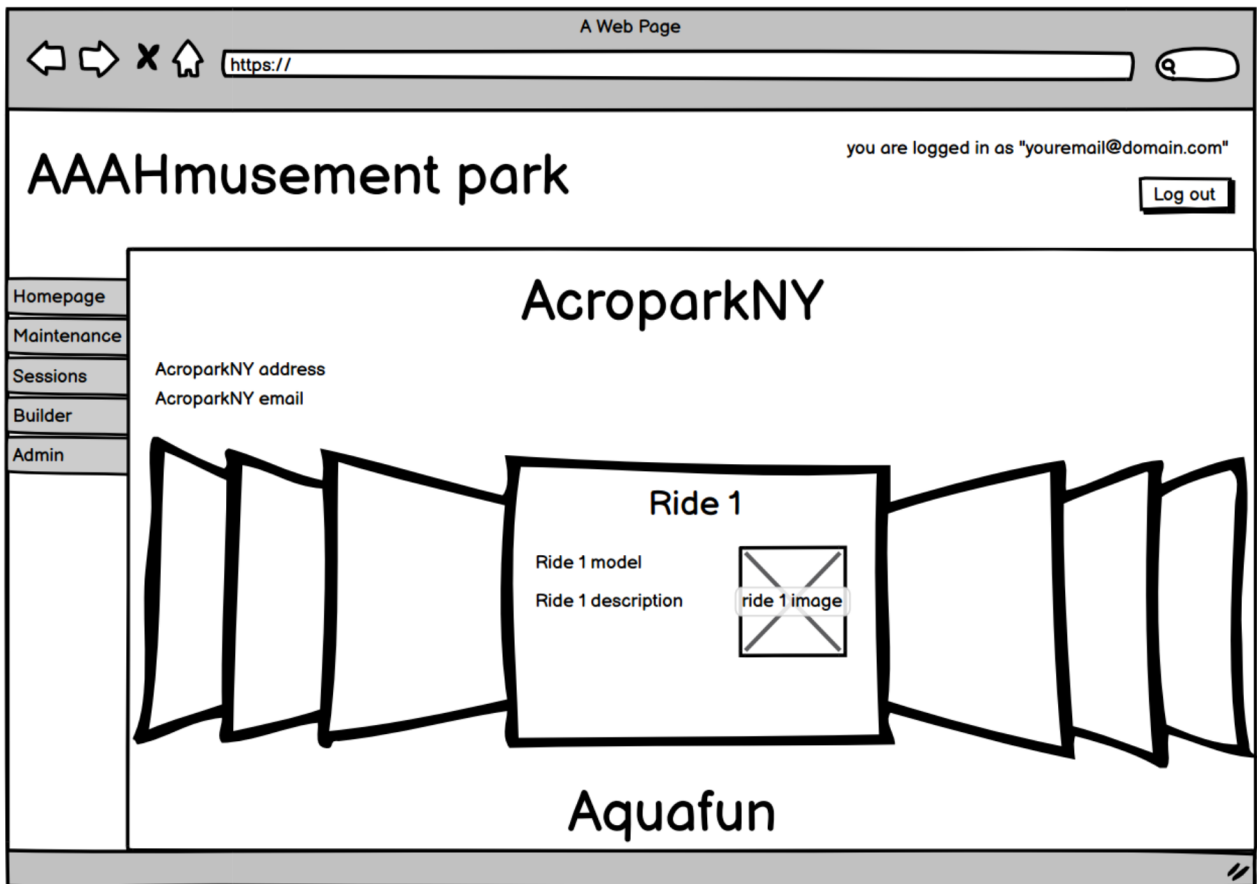
Presentation Logic Layer

The website will be divided into the following pages:

- Homepage: contains information on previous constructions made by the ride constructor. Developed via jsp.
- Login page: allows the users login to the web application. Developed via jsp.
- Registration page: allows the user to register to the web pages.
- Search maintenance page: allows users to search for maintenance events. written in HTML with the support of javascript.
- Insert maintenance page: allows to insert a new maintenance event. Written in HTML.
- Insert measurement file page: allows to upload a file containing the measurements. Written in HTML.
- Builder page: allows to redirect users toward pages handling specific resources. Written in HTML.
- 2 pages each to handle insertion and update of parks, rides and models. Written in HTML.
- Administration page: allows to administrate users' accounts. Written in HTML.
- A support page which displays messages upon completion of specific tasks, such as the removal or update of resources.

Homepage

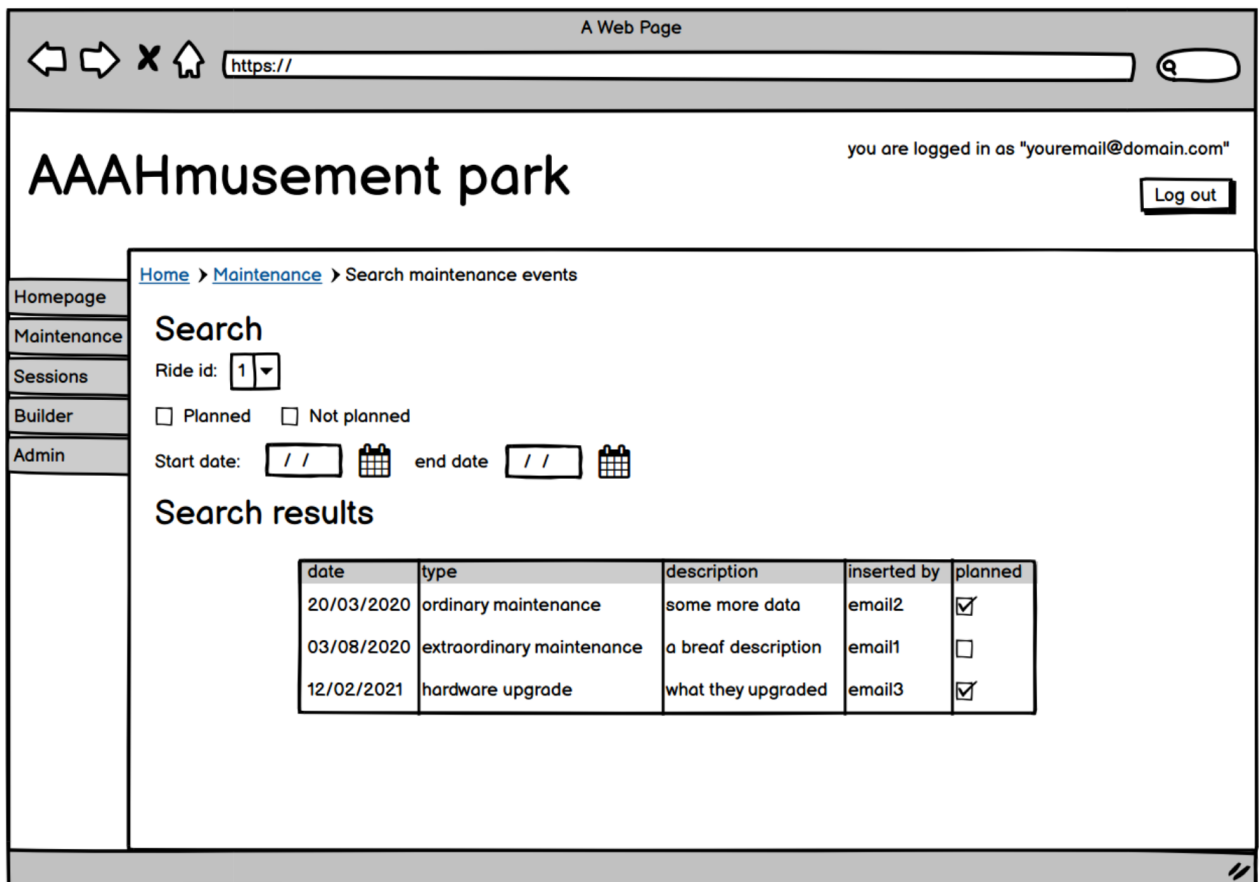
The homepage contains the information on already deployed rides and parks in which such rides have been deployed. In particular, the main area of the homepage will contain a section for each park. In this section the address and the email of the park are reported, so that visitors of the website can reach the park and experience by themselves the rides. Besides address and email, each section of each park will contain a carousel with rides for that park. The carousel will be organized in cards: each card contains pieces of information on the rides. In particular, it contains the description of the model of the ride, the description of the ride itself and an image representing the ride.



Search Maintenance Page

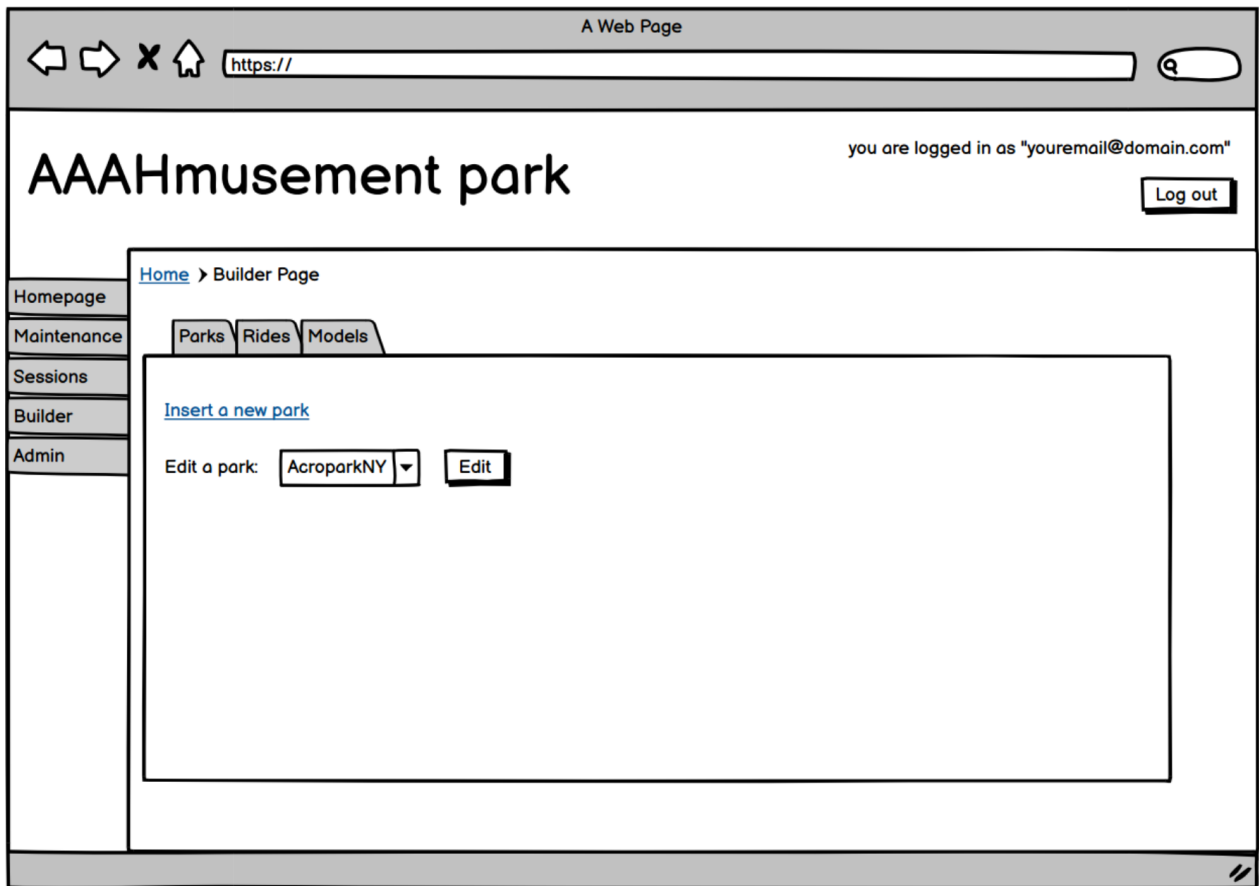
The “search maintenance page” contains a form used to filter out maintenance events. The user can select the ride to which the searched maintenance events refer to. They can filter events with two checkboxes to select either only events planned, not planned or both.

There are also two date input fields, they can be used to filter maintenance events in order to select only maintenance events occurring between two specific dates.



Builder Page

The general builder page is organized into folders. Each folder corresponds to a specific entity in the dataset, among those that can be managed by the builders. In particular, we have a folder for the parks, one for the rides and one for the models. By clicking on each folder, the user is presented with the interface to interact with the requested resource. In particular, there is a link that allows the user to reach the page for inserting a new resource and a selector that allows the user to select a specific resource based on its unique identifier and reach the page to edit it.



Insert Ride Page

The Insert Rides Page contains the form necessary to insert a new ride.

There is a static and a dynamic part of the page. In the static part, it is possible to insert the description of the ride, the park it is going to be deployed in and its model. Both the park input and the model input are going to be filled automatically, based on the data available in the database.

Dynamic aspects of the page regard the possibility of adding (and removing) on the fly additional devices that are mounted on the ride. By clicking on a "+" button, a new device form will be inserted in the device form area. There is also a "-" button, which will allow the user to remove devices. Note that, each device form will have its own "-" button. This way, the user can remove specific devices, without the need for additional effort that might stem from the removal of the last inserted device.

Upon completion, if the insertion ended correctly, the user is redirected to the homepage.

The screenshot shows a web browser window titled "A Web Page" with a URL bar containing "https://". The page header displays "AAAHmusement park" and a login status "you are logged in as 'youremail@domain.com'" with a "Log out" button. A left sidebar contains navigation links: "Homepage", "Maintenance", "Sessions", "Builder" (highlighted), and "Admin". The main content area shows a breadcrumb trail "Home > Builder Page > Insert a new ride". Below this, the "Ride" section contains a "Description" text box with placeholder text "A breaaf description of the ride", a "Park" dropdown menu with "AcroparkNY" selected, and a "Model" dropdown menu with "Carousel" selected. The "Devices" section, titled "Device 1", includes a "Name" text box with "dev1-name", a "Type" dropdown menu with "dynamometer" selected, and a "Description" text box with "some data about the device". There are "+" and "-" buttons below the device information.

Admin Page

The admin page is divided into three sections: "search", "user details" and "update user details". Upon entering the page, only the "search" section will be filled.

The admin page allows to search for users via email. The selector is filled automatically with the emails of all the registered users. Once the email of the user has been specified, by clicking on the "Search" button, the two following sections ("user details" and "update user details") are automatically filled with the information on the users. In particular, in the section "user details" is reported a table with the user email, first name, last name and role.

The section "update user details" contains a form, automatically filled with the data on the searched user. The admin can update such data. They can then click on either the button "update" which updates the user details, or the button "delete" which deletes the user from the database, revoking their possibility to login and therefore to access reserved areas of the website.

A Web Page

https://

AAAHmusement park

you are logged in as "youremail@domain.com"

Log out

Homepage

Maintenance

Sessions

Builder

Admin

Home > Admin

Search a user

Some user email

Search

User Details

email	First name	Last name	Role
guglielmo.faggioli@parksville.com	Guglielmo	Faggioli	admin

Update User Details

First name

Guglielmo

Last name

Faggioli

Role

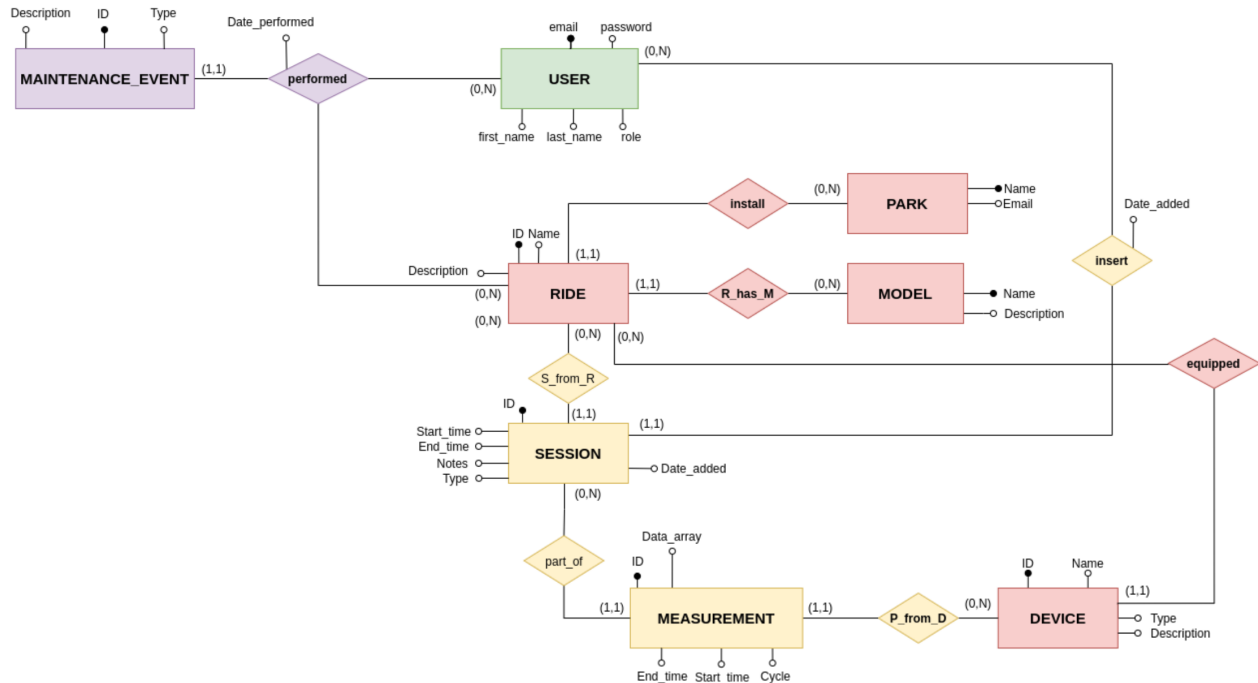
admin

Update User

Delete User

Data Logic Layer

Entity-Relationship Schema



The entity-relationship contains 8 main entities:

- **Maintenance_event**: describes maintenance events recorded by the fictitious ride constructor. The primary key is an integer with auto-increment. There are two additional attributes: the description, which is a CHAR field of variable length and the type, which is of type event categories (custom enumeration). To accountability reasons, the email of the user inserting the maintenance is also recorded. This corresponds to the relation “performed” in the ER schema. By being a 1-N relation, this information is recorded directly in the maintenance_event tuple.
- **User**: each user has, as primary key, their email, which is of type char with variable length (up to 64 characters). For each user we also record their first name, last name (of type char), role (of type roles) and password. Note that the password is hashed through md5 before storing it.
- **Park**: each park is uniquely identified by its name and has 2 attributes: its address and its email (used to contact the staff of the park).
- **Model**: The models of rides developed by the fictitious corporation. Each model is uniquely identified by a name and has a textual description.
- **Ride**: the main entity of the database. Its primary key is an integer incremented automatically upon insertion. It has an additional attribute which is its description. Each ride is deployed in a specific park,

(determining a relation N-1 with the parks) and is of a specific model (determining a relation N-1 with the models). The park in which the ride is deployed and its model are therefore memorized as external keys directly on the ride entity instances.

- Device: each ride has multiple devices, which are used to collect numerical data on the functioning of the ride itself. Each device is identified uniquely based on an auto increment integer. Each device is characterized by a name and a description. Finally, since they are in relation N-1 with the rides, they contain directly the id of the ride they are mounted on. note that the type of the device of type device types, which is a custom enumeration data type.
- Session: measurements are organized in sessions. Each session contains several measurements, for example one (or more, according to the number of devices mounted on the ride) for each cycle of the ride. Each session is identified uniquely through an integer with auto increment. Other attributes are the start and end time, represented through timestamps, the date it was added, the type of session and a free text field which contains additional descriptive notes. Each session is linked to a specific ride and thus it contains as a foreign key the id of the ride.
- Measurement: this table has two external keys, in particular, it refers to a specific session and a specific device. Each tuple contains information on the measurements collected by a specific device, during a specific session of measurements. There is an attribute, data_array, which contains the array of the measurements collected by the device. There are three more attributes, start_time and end_time which contain respectively the timestamp of the starting moment and ending moment of the measurements, and the attribute cycle, which indicates, in the session, the progressive number of cycles of the measurement.

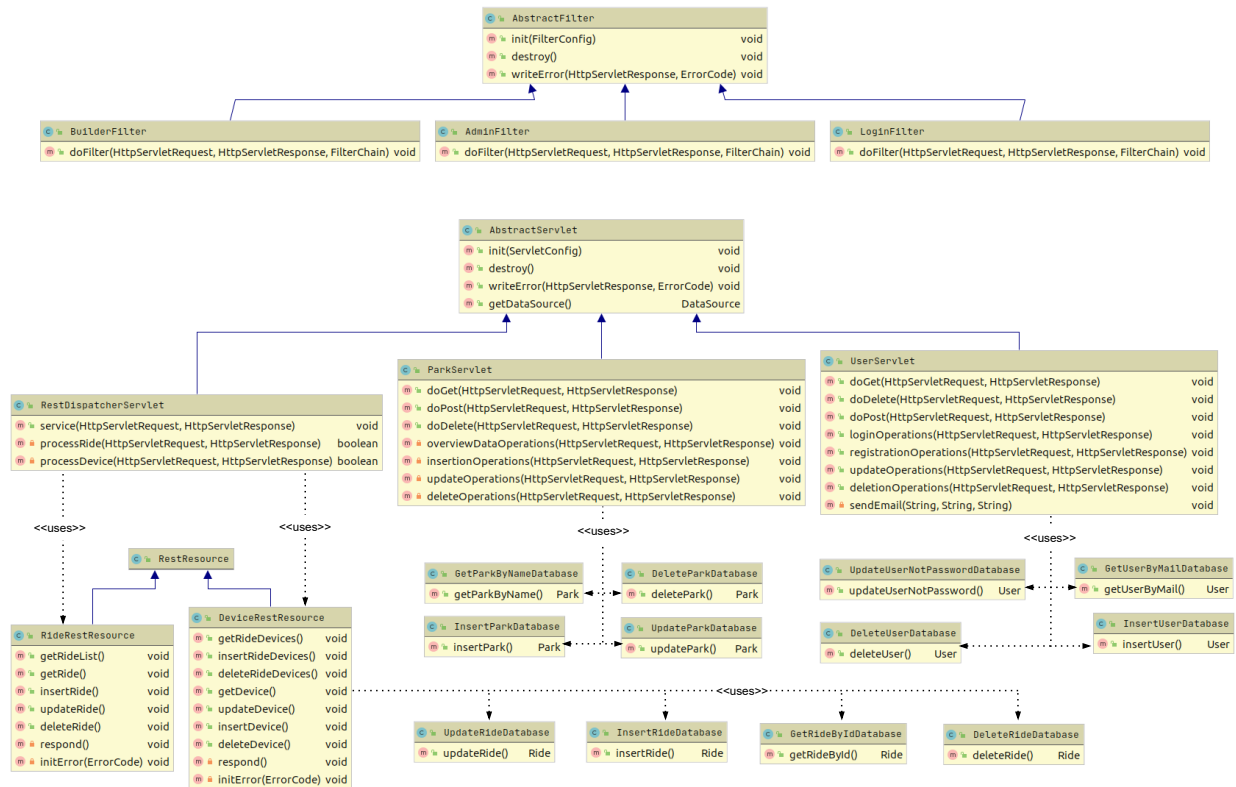
Other Information

It is possible to interact with most of the entities (insertion, deletion and update) directly via interface. The only exception is for the session and measurement entities, which are filled through files uploaded via interface.

There are 4 custom enumerations: eventcategories, which contain a value for each possible type of maintenance events. Indicates whether they were ordinary maintenance events, extraordinary and so on. roles (contains a value for each possible role of the users). devicetypes: contains a value for each possible device type (i.e., thermometer, dynamometer, barometer ...). sessiontypes: contains a value for each type of session (whether the session was a testing, monitoring, or ordinary session).

Business Logic Layer

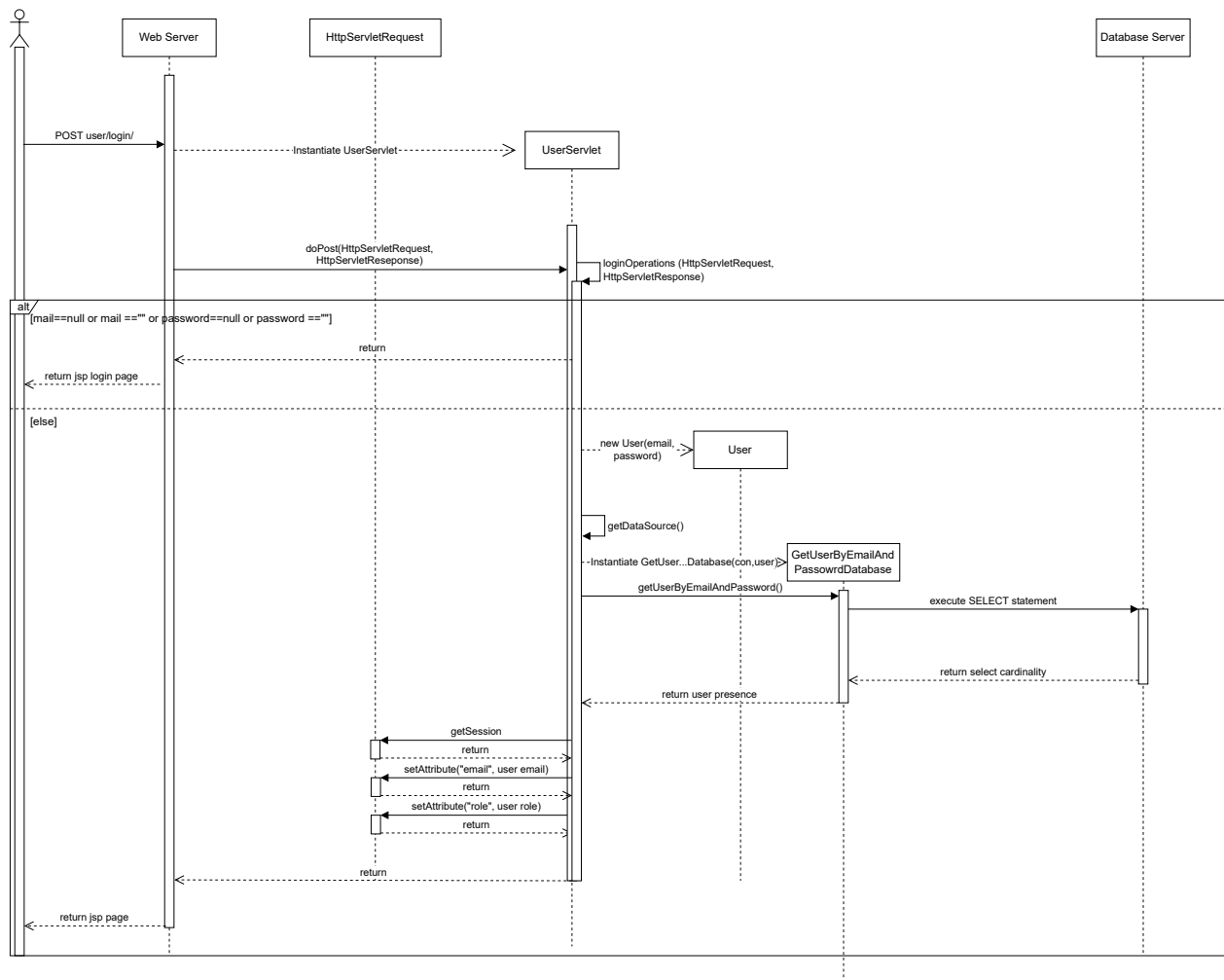
Class Diagram



The class diagram contains (some of) the classes used to handle three types of resources: users, rides and parks. It is possible to observe that we have a single servlet to handle the parks. Such servlet implements the `doGet` and `doPost` methods and is a subclass of the `HttpServlet` class. Concerning the rides, the servlet that implements the required behaviours to handle such resources is a more traditional servlet, based on the REST paradigm. In particular, we have a servlet, which also handles the Devices resource, which parses the URI, determining the type (and possibly the id) of the resource that the user wants to interact with. Once the servlet has processed the request, it forwards it to the proper Rest manager class. There are two rest manager classes: `RideRestResource` and `DeviceRestResource`. Both are sub-classes of `RestResource` and implement the methods to handle the proper resource. Concerning the user, there are multiple servlets used to handle the user resource. In particular, we have a servlet (`UserServlet`) which is mapped to URLs which are freely accessible from outside. This servlet is used to login and register new users. It has a subclass, `UserServlet`, which overrides the method "RegistrationOperations", by also sending an email to the user upon registration. Note that `UserServlet` has been kept mainly for legacy reasons. Additionally, the `UserServlet` implements methods which are accessible only to users with the "admin" authorization level. In particular, it is responsible for the retrieval, update and deletion of users. Several Data Access Objects (DAOs) allow us to obtain different resources. We report the

Classes that allow to handle objects of different types.

Sequence Diagram



Here reported the sequence diagram for the login operations. The user executes a POST request to the web server, specifying the URI user/login/. Additionally, the data about the user which is trying to login (name and password) are passed to the web server. The web server instantiates the UserServlet and calls its doPost method, passing the HttpServletRequest and the HttpServletResponse. The UserServlet analyzes the request and recognizes that it is a login operation, thus it calls its method loginOperations. Such method controls whether the email and the password have been correctly provided, if not, returns a the login.jsp page, showing the error. Else, a new User object is initialized with the provided data. After that, a DAO called GetUserByEmailAndPasswordDatabase is initialized, using a connection and the User. Calling the method getUserByEmailAndPassword we verify if it is possible to retrieve a user given the specific email and password.

If so, then the user is present in our database and can be logged in. The control is returned to the `UserController`, which obtains the session from the `HttpServletRequest` and sets two parameters in the session: the email of the user and their role. Finally, the `homepage.jsp` page, instantiated with the user's data, is provided back to the actor.

REST API Summary

The rest API is studied to experiment with multiple different approaches to the REST paradigm. More in detail, endpoints associated with rides and devices are developed in a more traditional way, with the entirety of the information needed to satisfy the request directly in the URI. Other endpoints require additional information (such as the type of operation required) as additional parameters in the request.

Part of the URIs are filtered through different filters. M indicates that only users with the role of maintainer can access to the endpoint, B indicates that only builders can access the endpoint, A that only administrators can request the specified URI to the server.

URI	Method	Description	Filter
<code>maintenance/search/</code>	GET	By specifying search parameters directly in the URL it is possible to search for specific maintenance events.	M
<code>maintenance/insert/</code>	POST	Allows to insert new maintenance events	M
<code>measurement/upload/</code>	POST	Allows to upload a measurement file	M
<code>model/editable/</code>	GET	Returns a ride model, given its name as parameter in the URL	B
<code>model/list/</code>	GET	Returns the list of all models available in the database	B
<code>model/insert/</code>	POST	Allows to insert a new model, passed as additional data of the request	B
<code>model/update/</code>	POST	Allows to update a model passed as additional data in the request	B
<code>model/</code>	DELETE	Allows to delete a model of rides	B
<code>park/editable/</code>	GET	returns the data for a specific park, given the name as URL parameter.	B
<code>park/list/</code>	GET	Returns the list of all parks in the database	B
<code>park/overview/</code>	GET	Returns the structure used to initialize the homepage.	
<code>park/insert/</code>	POST	Allows to insert a park passed as data	B
<code>park/update/</code>	POST	Allows to update a park passed as data	B
<code>park/</code>	DELETE	Allows to delete a park passed as data	B
<code>protected/user/</code>	GET	Allows to get information on a specific user	A
<code>protected/user/</code>	POST	Allows to update the information on a specific user	A
<code>protected/user/</code>	DELETE	Allows to delete a user	A

schema/roles/	GET	Allows to get schema information on the types of role available in the database	A
schema/devicetypes/	GET	Returns the schema information on the types of the devices that have been defined in the schema	B
schema/eventcategories/	GET	Returns the schema information on the event categories that have been defined in the schema	M
schema/sessiontypes/	GET	Returns the schema information on the types of the session types that have been defined in the schema	M
user/	GET	if the additional parameter "operation" is passed and is set to "logout" allows to logout a user.	
user/login/	POST	Allows to pass to the web server the credentials of a user and checks if the user is in the database and their mail and password correspond.	
user/register/	POST	Allows to register a new user.	
ride/	POST	Allows to insert a new ride.	B
list/ride/	GET	Allows to get the list of all rides available in the database.	B
ride/{rideid}	GET	Allows to get information on a ride	B
ride/{rideid}	PUT	Allows to update ride informations	B
ride/{rideid}	DELETE	Allows to delete a ride	B
device/ride/{rideid}	GET	Allows to get the list of all devices associated to the ride {rideid}	B
device/ride/{rideid}	POST	Allows to insert at once the a of devices (passed as data) and associate them with the ride identified by {ride id}	B
device/ride/{rideid}	DELETE	Allows to delete all the devices associated with the ride {rideid}	B
device/{deviceid}	GET	Allows to get information on a specific device identified by the id {deviceid}	B
device/{deviceid}	PUT	Allows to update the information of a specific device, identified by the id {deviceid}	B
device/{deviceid}	DELETE	Allows to delete a specific device	B
device/	POST	Allows to insert a new device, as specified in the data field of the header.	B

Table 2: Main REST API endpoints

REST Error Codes

Here the list of errors defined in the application. Application specific errors have the application error which follows a progressive numeration starting from -100. METHOD NOT ALLOWED errors are identified with the error code -500. Internal errors, which correspond to crashes, servlet exceptions, or problems with the input/output streams are identified with the Error Code -999.

Error Code	HTTP Status Code	Description
-100	BAD_REQUEST	Wrong Park id format.
-101	NOT_FOUND	The user is looking for a park that was not found.
-102	BAD_REQUEST	The user tried to register without specifying all the details. Used also with forms that need to be fully filled.
-103	BAD_REQUEST	The user tried to login without specifying the Email.
-104	BAD_REQUEST	The user tried to login without specifying the password.
-105	BAD_REQUEST	User submitted wrong credentials in the login form.
-106	NOT_FOUND	User to be deleted not found.
-107	BAD_REQUEST	different passwords when repeating the password in the registration phase.
-108	CONFLICT	mail already used.
-109	BAD_REQUEST	Unrecognized role.
-110	CONFLICT	The park name has already been used.
-111	CONFLICT	The model name has already been used.
-112	NOT_FOUND	Model not found.
-113	BAD_REQUEST	Wrong rest request format.
-114	CONFLICT	This ride id is already present.
-115	NOT_FOUND	Ride not found.
-116	CONFLICT	Ride requested and ride provided do not match.
-117	NOT_FOUND	User not found.
-118	BAD_REQUEST	Ride id missing.
-119	NOT_FOUND	Device not found.
-120	BAD_REQUEST	The input json is in the wrong format.
-200	BAD_REQUEST	Operation unknown.
-500	METHOD_NOT_ALLOWED	The method is not allowed.
-999	INTERNAL_SERVER_ERROR	There was an internal error that the server was not able to manage.

Table 3: Error codes for the REST interface of the Amusement Park application back-end

REST API DETAILS

We report here 3 different resource types that our web application handles during its functioning.

Homepage content

The homepage contains an aggregation of several pieces of information. In particular, it contains the full list of

parks in the database, for each park contains the list of rides that have been deployed by the fictitious corporation in such a park, and for each ride contains the description of the model.

- URL: `park/overview/`
- Method: GET
- URL Parameters: No parameters are required in the url
- Data Parameters: No data passed
- Success Response:
Upon success, the servlet returns a JSON object with the required information.
- Error Response:
Code: 500 INTERNAL_SERVER_ERROR
Content: `{'error': {'code': -999, 'message' : 'Internal Error.'}}`
When: if there is a `SQLException` or a `NamingException`, this error is return

Ride

The following endpoint allows to insert a new ride into the database.

- URL: `ride/`
- Method: POST
- URL Parameters: No parameters are required in the url
- Data Parameters:
Required:
 - `parkdid = {string}`: The name of the park the ride is deployed in. As an additional requirement, it should belong to parks already inserted in the dataset.
 - `modelid = {string}`: The id of the model of the ride. As an additional requirement, it should belong to models already inserted in the dataset.

Optional:

- `description = {string}`: The description of the ride
- Success Response:

Code: 200

Content: `{data: {rideid : 12, parkid: 'AcroParkNY', modelid: 'rollercoaster' }}`

- Error Response:

Code: 404 NOT_FOUND

Content: {`''error''`: {`''code''`: -101, `''message''` : `''Park not found.'''`}}

When: The user has passed a park id which was not found in the database.

Code: 404 NOT_FOUND

Content: {`''error''`: {`''code''`: -112, `''message''` : `''Model not found.'''`}}

When: The user has passed a model id which was not found in the database

Code: 500 INTERNAL_SERVER_ERROR

Content: {`''error''`: {`''code''`: -999, `''message''` : `''Internal Error.'''`}}

When: if there is a `SQLException` or a `NamingException`, this error is return

Code: 400 BAD_REQUEST

Content: {`''error''`: {`''code''`: -120, `''message''` : `''The input json is in the wrong format.'''`}}

When: The user has passed in the data payload a json ill-formatted or without all the required fields.

Code: 401 UNAUTHORIZED

When: The uri is accessible only to logged users with at least the role "builder"

Users

The following endpoint allows to register a new user.

- URL: `user/register`
- Method: `POST`
- URL Parameters: No parameters are required in the url
- Data Parameters:

Required:

- `email` = {string}
- `password` = {string}
- `rpassword` = {string}: need to coincide with the password parameter.
- `first_name` = {string}
- `last_name` = {string}

- Success Response:

Code: 200

Content: the user is redirected to the login page, which in turns uses a filter to create a session for the user and redirect them to the homepage.

- Error Response:

Code: 400 BAD_REQUEST

Content: `{'error': {'code': -102, 'message' : 'One or more input fields are empty.'}}`

When: some of the parameters are missing or are empty string.

Code: 400 BAD_REQUEST

Content: `{'error': {'code': -108, 'message' : 'Different Passwords.'}}`

When: parameters password and rpassword are different.

Code: 409 CONFLICT

Content: `{'error': {'code': -107, 'message' : 'Mail Already Used.'}}`

When: the user is trying to register with a mail already present in the database.

Code: 500 INTERNAL_SERVER_ERROR

Content: `{'error': {'code': -999, 'message' : 'Internal Error.'}}`

When: if there is a `SQLException` or a `NamingException`, this error is return.

Group Members Contribution

Guglielmo Faggioli: implementation of the following DAOs: ... implementation of the following servlets: ...
implementation of the following pages: ... preparation of the following schemas: